

Aug 30 2024

Physical database design

Text

A logistic company records data about drivers and trips.

- **Table** DRIVER(DriverID, LastName, Name, City, ...) contains 50K tuples over 5K blocks, with a primary hash structure built on DriverID and filling factor below 50%.
- **Table** TRIP(TripID, Date, OriginID, DestID, driverID, TotalKM) records 15M tuples, primarily stored sequentially ordered by DriverID on 150K blocks.
- We know that $\text{val}(\text{OriginID})=5\text{K}$ and $\text{val}(\text{City})=25$.

Describe briefly (but precisely) a reasonably efficient query plan and estimate the execution cost for the following query in the three scenarios below. If a plan uses the cache, the same strategy without caching must be described too.

```
SELECT *  
FROM DRIVER  
WHERE City="Milan" AND DriverID IN  
SELECT DriverID  
FROM TRIP  
WHERE OriginID= "Linate"
```

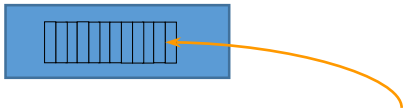
Text

1. Illustrate the three cases: nested loop, a plan starting from the scanning of TRIP and a plan starting from the scanning of DRIVER
2. Table `TRIP` table has a secondary hash index built on `DriverId`, with the same hash function used for the primary structure of the `DRIVER` table, with no overflow cost and 5K blocks.
3. Table `TRIP` also has a B+ index built on `OriginID` as search key, with 3 intermediate levels and 50K leaf nodes.

Data structures and statistics /scenario a

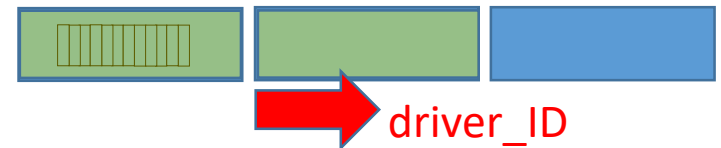
```
SELECT *  
FROM DRIVER  
WHERE City="Milan" AND DriverID IN  
(SELECT DriverID  
FROM TRIP WHERE OriginID= "Linate")
```

DRIVER(DriverID, LastName, Name,
City, ..)



- 50K tuples H(DriverID)
- 5K blocks
- filling factor < 50%
- Val(City)=25
- 2k Milan drivers

TRIP(tripID,date,
OriginID,driverID,TotalKM)



- 15M tuples
- Sequentially ordered by DriverID
- 150K blocks (100 trips per block)
- val(OriginID) = 5k
- 3k trip per OriginID
- 300 trips (15M/50k) per driver
- 3 trip blocks per driver

Scenario a

- Nested loop
 - Cost = $5k \times 150k = 750M$
- Scan DRIVER, filter and lookup TRIP
 - Scan driver: 5k
 - For all 2k Milan tuples, lookup trip: $2k \times 75k$ (on average for each driver one has to find the first block, fetch the 3 consecutive blocks and then stop; this procedure scans half of the blocks of the TRIP table on average)
 - Without assumptions on the drivers distribution in TRIP, lookup the entire TRIP every time $\rightarrow 2k \times 150k = 300M$
 - Cost = $150M$
- Scan TRIP filter and lookup DRIVER
 - Scan trip: $150k$
 - For all 3k trips of Linate
 - Get DriverID and lookup DRIVER via hash to check city = Milan: $3k \times 1$
 - Cost: $150k + 3k = 153k$

Data structures and statistics /scenario b

```
SELECT *  
FROM DRIVER  
WHERE City="Milan" AND driverID IN  
(SELECT driverID  
FROM trip WHERE OriginID= "Linate")
```

driver(driverID, LastName, Name,
City, ..)

trip(tripID, date,
OriginID, driverID, TotalKM)

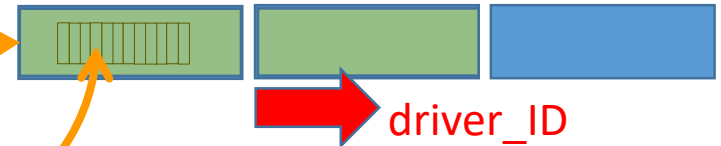


- 50K tuples H(driverid)
- 5K blocks
- filling factor < 50%
- Val(City)=25
- 2k Milan drivers

300 trips per
driver



- DriverID
- 5k buckets
- 1 i/o access cost



- 15M tuples
- Sequentially ordered by DriverID
- 150K blocks (100 trips per block)
- val(OriginID) = 5k
- 3k trips per OriginID
- 300 trips (15M/50k) per driver
- 3 trip blocks per driver

Scenario b

- Scan DRIVER and lookup TRIP
 - No caching
 - Scan DRIVER: 5K
 - For all 2k Milan drivers access hash and follow 300 pointers: $2k \times (1 + 300) = 602k$
 - Cost = **607k**
 - Caching of 3 contiguous blocks of TRIP of each driver
 - Scan driver: 5K
 - For all 2k Milan drivers access hash (1 i/o) and fetch 3 primary blocks: $2k \times (3 + 1) = 8k$
 - Cost = **13 k**
- Compute HASHED JOIN between DRIVER and TRIP
 - Cost without caching:
 - 5k (primary hash on driver) +
 - Read all hash index buckets and for only the Milan DriverIDs access the 300 trips
 $5k + (2k * 300) = 5k + 600.000 = 605.000$
 - Cost = **610k**
 - Cost with caching:
 - 5k (primary hash on driver) +
 - Read all hash index buckets and for only the Milan driverIDs access the 300 trips but after reading a block for a trip cache it and for the next pointer to trip check the cached block before accessing the table data
 - $5k + (2k * 3) = (\text{caching the 3 contiguous blocks of tripS of each driver}) = 11 k$
 - Cost = **16k**

Data structures and statistics /scenario c

```
SELECT *  
FROM driver  
WHERE City="Milan" AND driverID IN  
(SELECT driverID  
FROM trip WHERE OriginID= "Linate")
```

driver(driverID, LastName, Name,
City, ..)

trip(tripID, date,
OriginID, driverID, TotalKM)



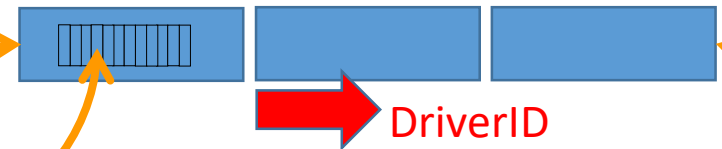
- 50K tuples H(driverid)
- 5K blocks
- filling factor < 50%
- Val(City)=25
- 2k Milan drivers

- DriverID
- 5k buckets
- 1 access cost

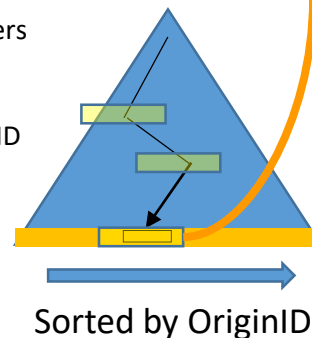
300 trip per
driver



- 15M tuples
- Sequentially ordered by DriverID
- 150K blocks (100 trips per block)
- val(OriginID) = 5k
- 3k trip per OriginID
- 300 trips (15M/50k) per driver
- 3 trip blocks per driver



OriginID
50k nodes
4 levels
300 pointers
per node
10 nodes
per OriginID



Scenario C

- Search TRIP via B+ tree and access driver for 3k trips of origin Linate
 - Access B+ = 3 (traversal) + 10/11 (leaves)
 - Access TRIP to get DriverID: 3k
 - Access driver via DriverID through primary hash and check city: 3k
 - Cost: 6013 or 6014